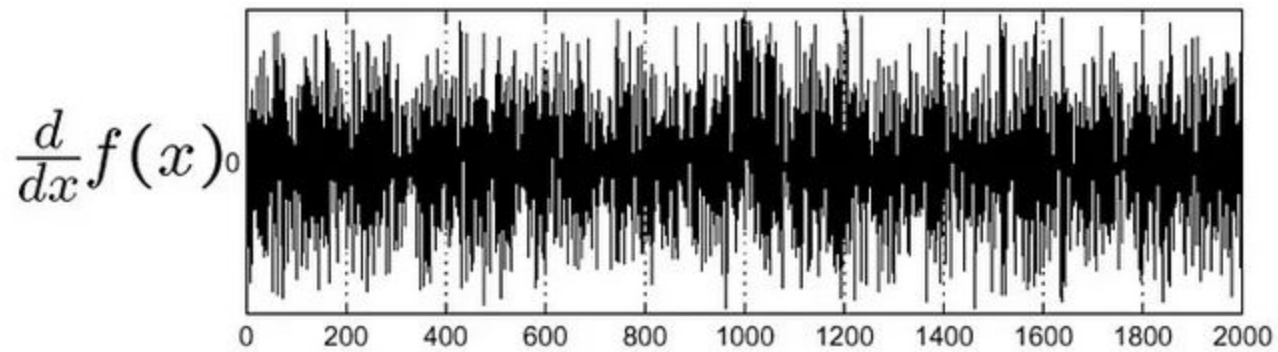
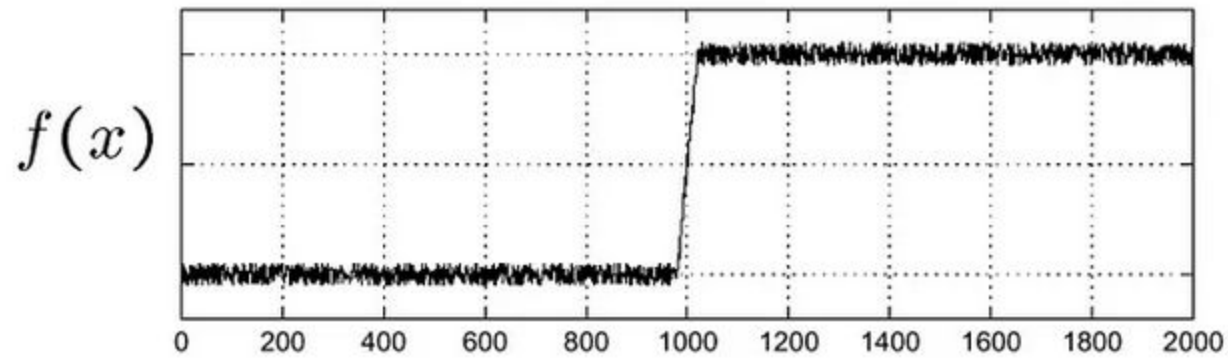


EE 604 Image Processing

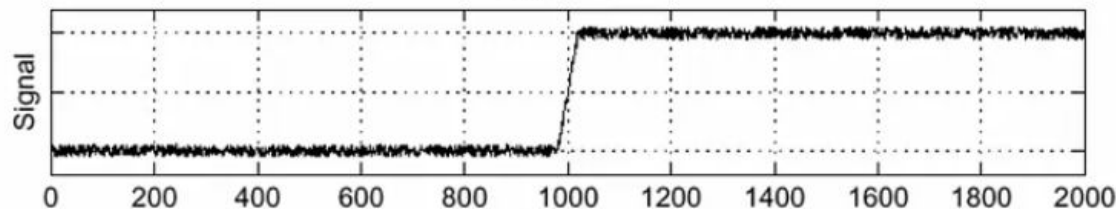
Lec 04: Spatial Domain - III

Koteswar Rao Jerripothula
Associate Professor
EE Department
IIT Kanpur

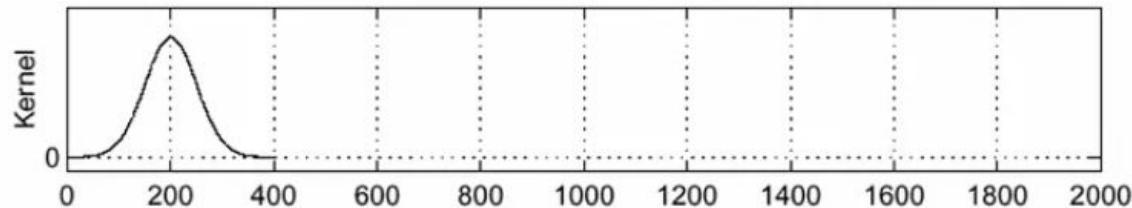


Better Approach

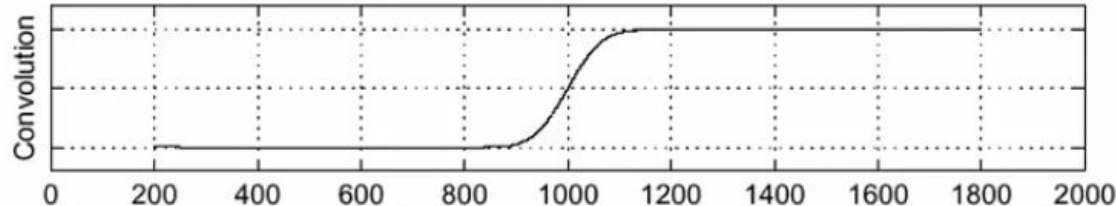
Image f



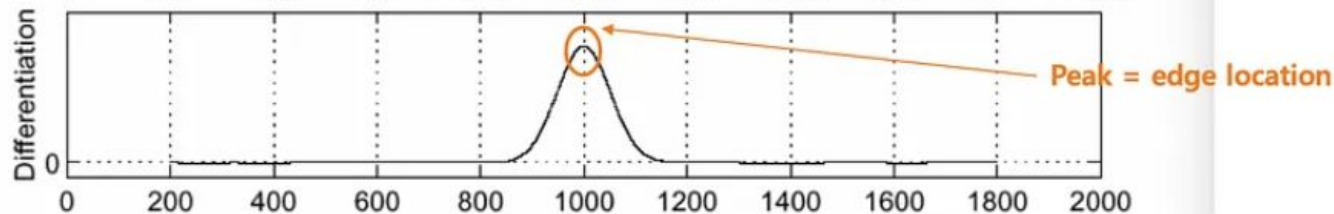
Gaussian Filter h



Convolution $h \star f$



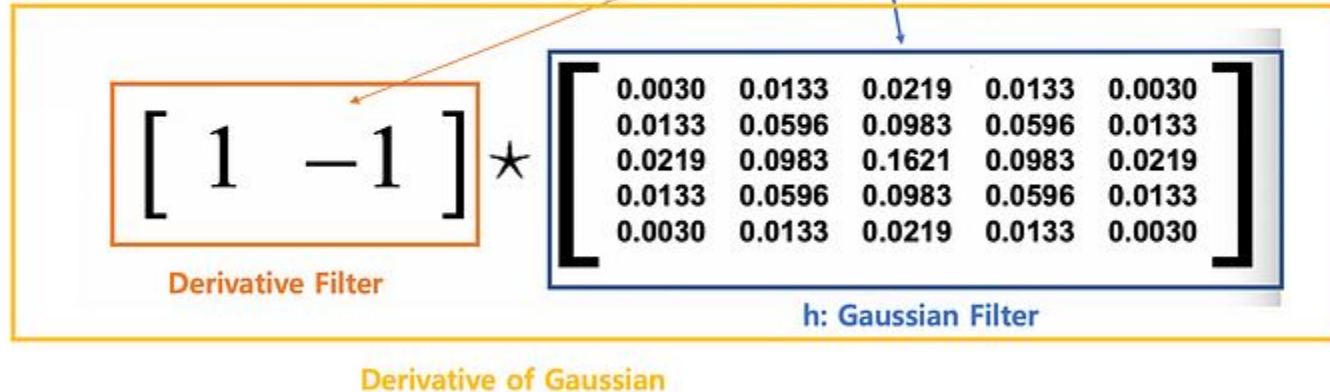
Derivative $\frac{\partial}{\partial x}(h \star f)$



Can we make it even better?

DoG: Derivative of Gaussian Filter

$$\frac{\partial}{\partial x}(h \star f) = \left(\frac{\partial}{\partial x}h\right) \star f$$

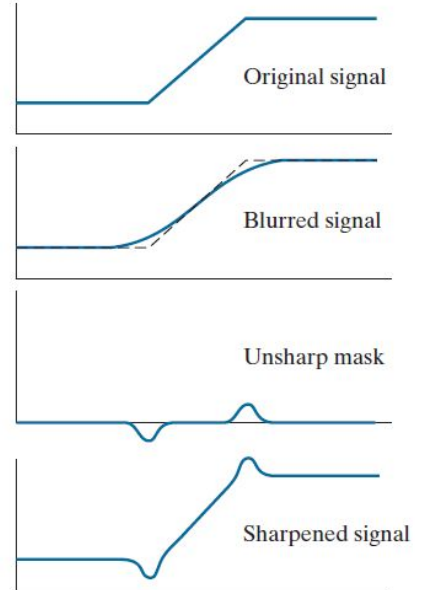


UNSHARP MASKING

1. Blur the original image.
2. Subtract the blurred image from the original. Let's call the resulting difference as mask.
3. Add the mask to the original.

a
b
c
d

1-D illustration of the mechanics of unsharp masking.
(a) Original signal.
(b) Blurred signal with original shown dashed for reference.
(c) Unsharp mask.
(d) Sharpened signal, obtained by adding (c) to (a).



Letting $\bar{f}(x, y)$ denote the blurred image, the mask in equation form is given by:

$$g_{\text{mask}}(x, y) = f(x, y) - \bar{f}(x, y)$$

Then we add a weighted portion of the mask back to the original image:

$$g(x, y) = f(x, y) + k g_{\text{mask}}(x, y)$$

When $k = 1$ we have unsharp masking

When $k > 1$, the process is referred to as highboost filtering.

Choosing $k < 1$ reduces the contribution of the unsharp mask.



a	b	c
d	e	

(a) Original image of size 600×259 pixels. (b) Image blurred using a 31×31 Gaussian lowpass filter with $\sigma = 5$. (c) Mask. (d) Result of unsharp masking using Eq. (3-56) with $k = 1$. (e) Result of highboost filtering with $k = 4.5$.

Edge detection using gradients

a
b c
d e

(a) A 3×3 region of an image, where the z s are intensity values.
(b)–(c) Roberts cross-gradient operators.
(d)–(e) Sobel operators. All the kernel coefficients sum to zero, as expected of a derivative operator.

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

-1	0	0	-1
0	1	1	0

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

$$M(x, y) = \left[(z_9 - z_5)^2 + (z_8 - z_6)^2 \right]^{1/2}$$

$$M(x, y) = \left[g_x^2 + g_y^2 \right]^{1/2} = \left[\left[(z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) \right]^2 + \left[(z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) \right]^2 \right]^{1/2}$$

For edge detection, considering the two responses (obtained using the two masks for a given pixel) as elements of a vector, compute the magnitude of the vector.

Laplacian Filter

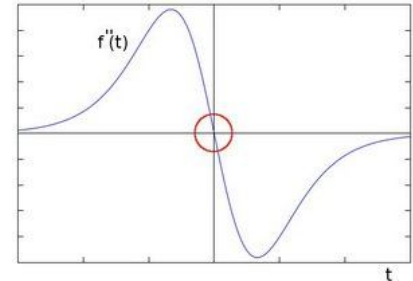
- Laplacian filter is one of edge detectors used to compute the second spatial derivatives of an image.
- Detects sharp changes like lines, edges, and isolated points
- Unlike first-order gradient operators that require separate horizontal and vertical calculations, it highlights regions of rapid intensity change in all directions at once by finding zero-crossings.

The mathematical definition of Laplacian is as follows.

$$\begin{aligned}\text{Laplacian } \nabla^2 f &= \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y} \\ &= f(x+1, y) + f(x-1, y) - 2f(x, y) + f(x, y+1) + f(x, y-1) - 2f(x, y) \\ &= f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)\end{aligned}$$

Given the definition, discretized 3 x 3 Laplacian filter (because we are dealing with images and they are discrete) for an image f is defined as an array below:

0	1	0
1	-4	1
0	1	0



Another version of Laplacian filter to make it isotropic (rotation-invariant).

1	1	1
1	-8	1
1	1	1

Image Sharpening

$$g(x, y) = f(x, y) + c \left[\nabla^2 f(x, y) \right]$$



Using 1st version of
Laplacian

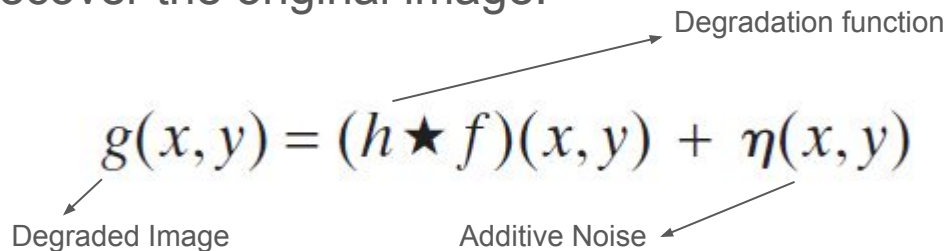


Using 2nd version of
Laplacian

Image Restoration

Image restoration means attempting to recover an image that has been degraded by using a priori knowledge of the degradation phenomenon.

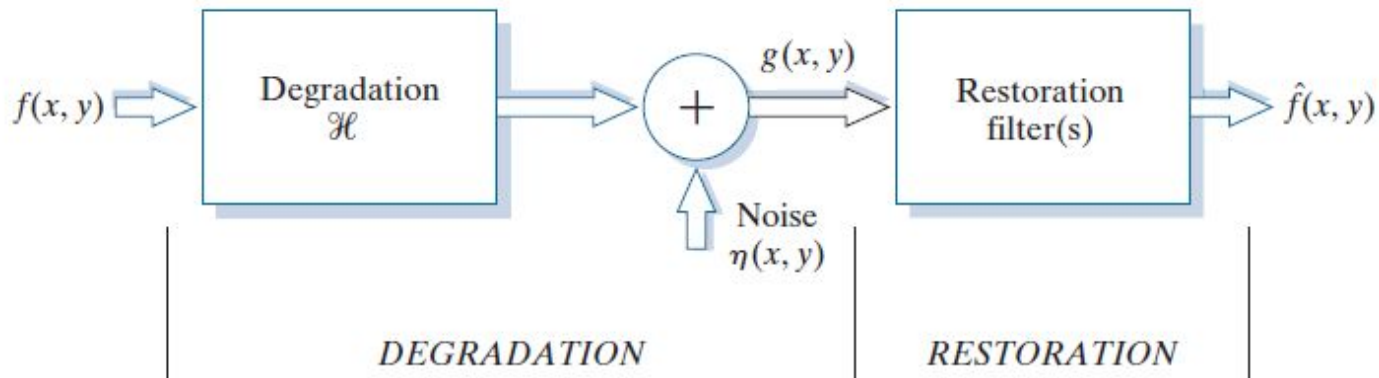
It involves modeling the degradation and applying the inverse process in order to recover the original image.

$$g(x, y) = (h \star f)(x, y) + \eta(x, y)$$


The diagram shows the equation $g(x, y) = (h \star f)(x, y) + \eta(x, y)$ with three labels and arrows pointing to its components: 'Degraded Image' points to $g(x, y)$, 'Degradation function' points to h , and 'Additive Noise' points to $\eta(x, y)$.

Image degradation model in spatial domain

A model of the
image
degradation/
restoration
process.



Noise Sources:

- Image Acquisition
 - Environmental factors
 - Quality of sensing elements
- Image Transmission
 - Interference in transmission channel

Assumptions:

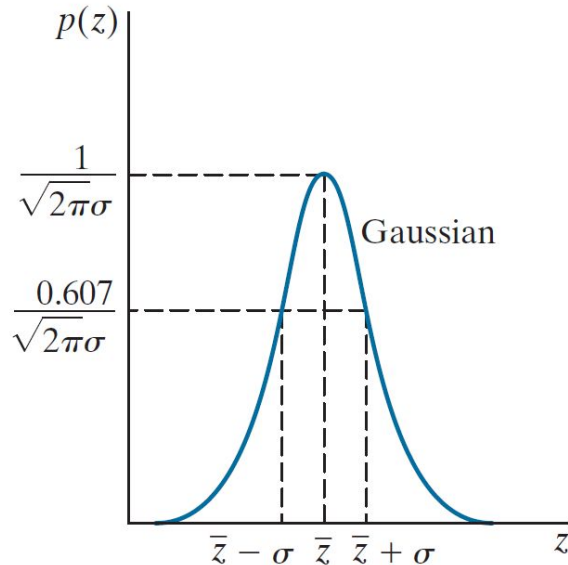
- Noise is independent of spatial coordinates
- No correlation between pixel values and the values of the noise component

(Knowing the value or location of a pixel in the original image gives **no information about the noise value** added to that pixel)

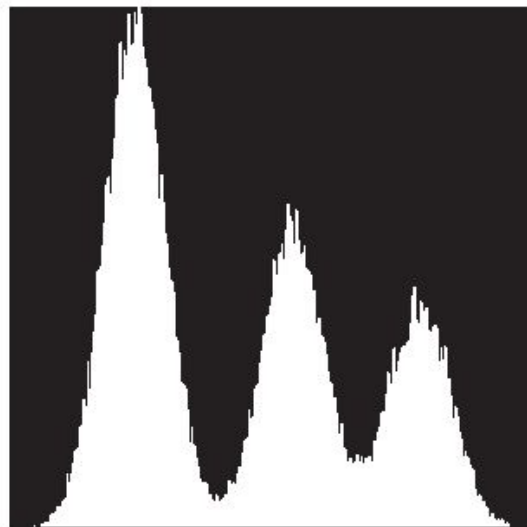
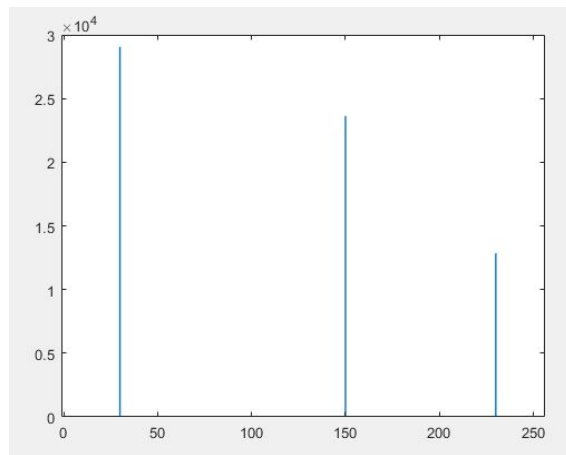
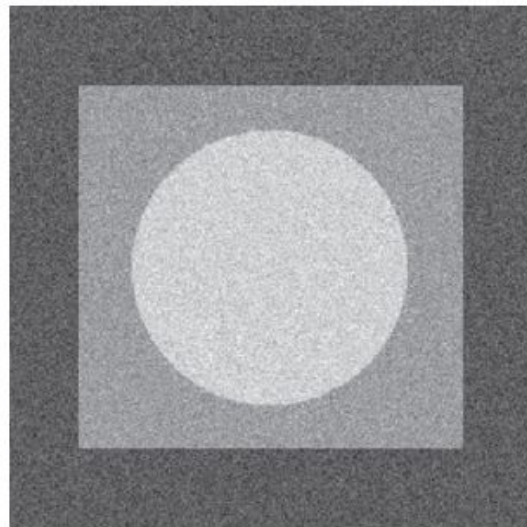
Gaussian Noise

The PDF of a *Gaussian* random variable, z , is defined by the following familiar expression:

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(z - \bar{z})^2}{2\sigma^2}} \quad -\infty < z < \infty$$



The PDF tells us the **probability of different noise values occurring**. Studying the PDF helps us understand **how the noise behaves statistically**, so that we can choose or design an appropriate restoration method.



Rayleigh Noise

The PDF of *Rayleigh* noise is given by

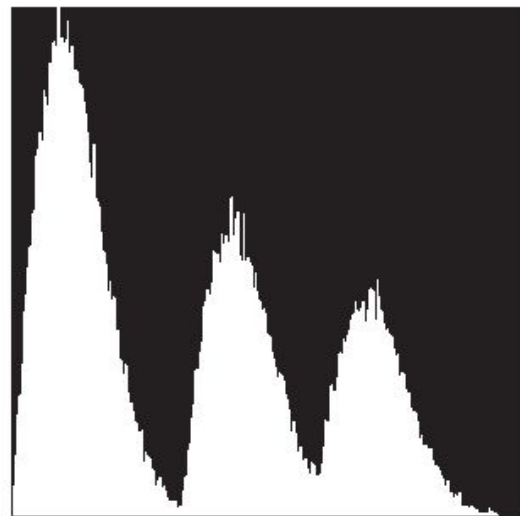
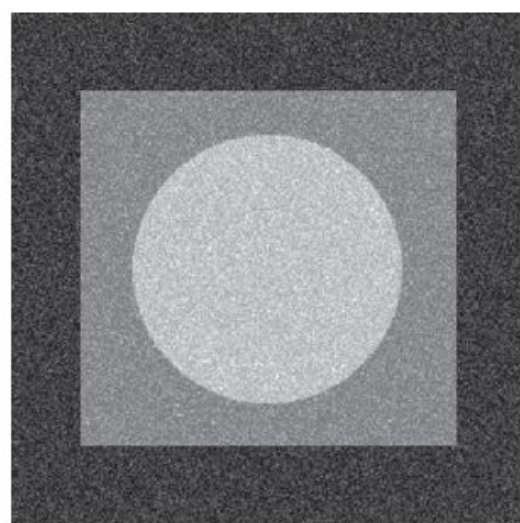
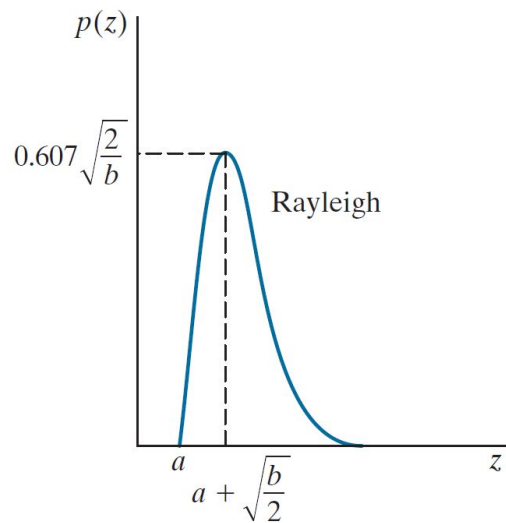
$$p(z) = \begin{cases} \frac{2}{b}(z - a)e^{-(z - a)^2/b} & z \geq a \\ 0 & z < a \end{cases}$$

The mean and variance of z when this random variable is characterized by a Rayleigh PDF are

$$\bar{z} = a + \sqrt{\pi b/4}$$

and

$$\sigma^2 = \frac{b(4 - \pi)}{4}$$



Erlang (Gamma) Noise

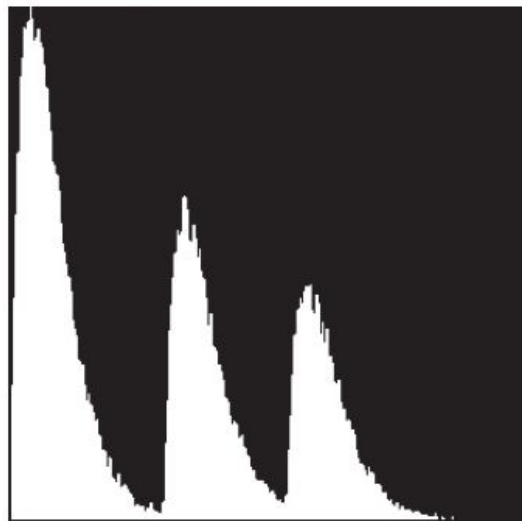
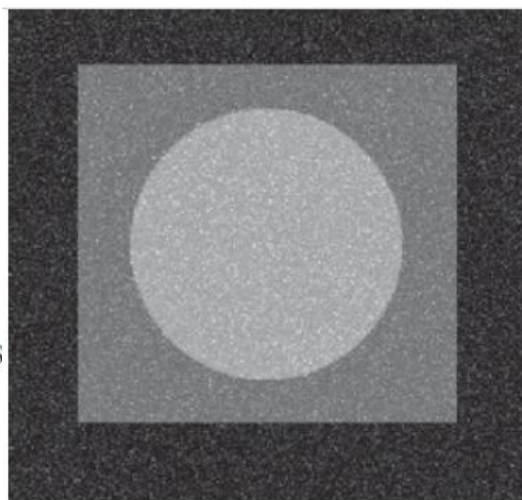
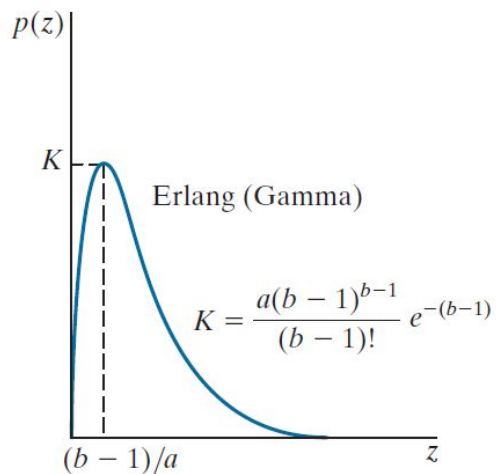
The PDF of Erlang noise is

$$p(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & z \geq 0 \\ 0 & z < 0 \end{cases}$$

where the parameters are such that $a > b$, b is a positive integer, and “!” indicates factorial. The mean and variance of z are

$$\bar{z} = \frac{b}{a}$$

$$\sigma^2 = \frac{b}{a^2}$$



Exponential Noise

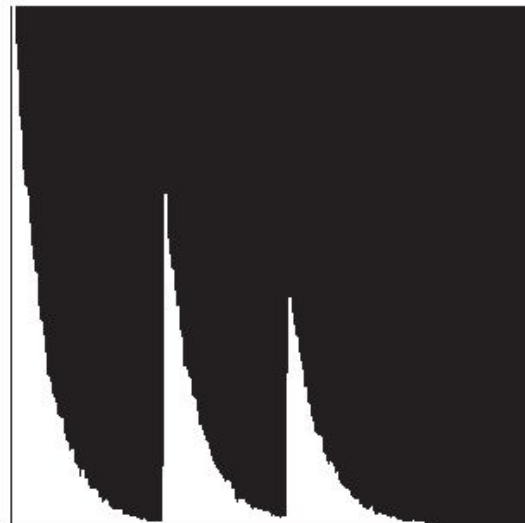
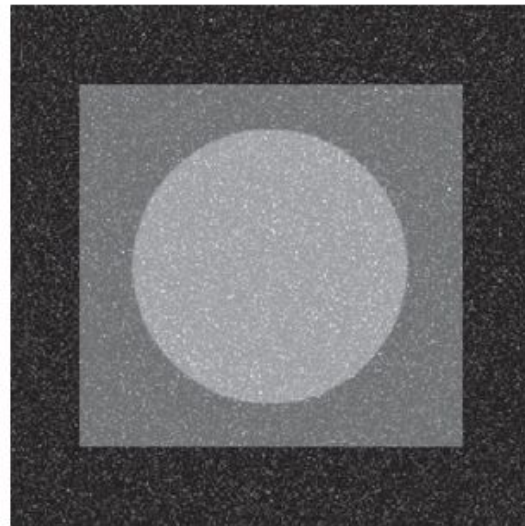
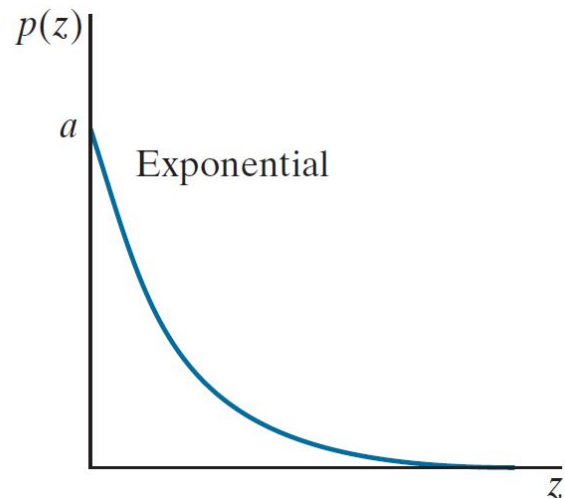
The PDF of *exponential* noise is given by

$$p(z) = \begin{cases} ae^{-az} & z \geq 0 \\ 0 & z < 0 \end{cases}$$

where $a > 0$. The mean and variance of z are

$$\bar{z} = \frac{1}{a}$$

$$\sigma^2 = \frac{1}{a^2}$$



Uniform Noise

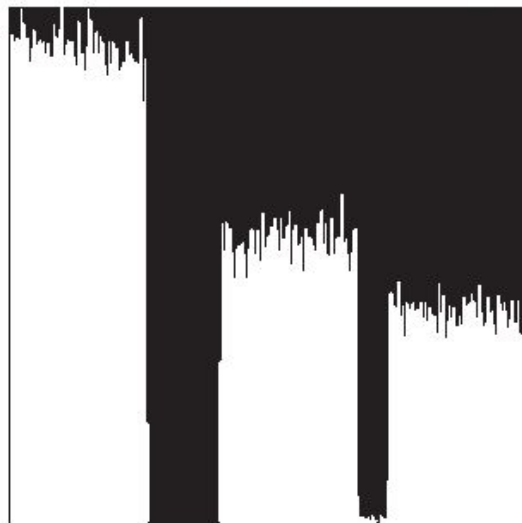
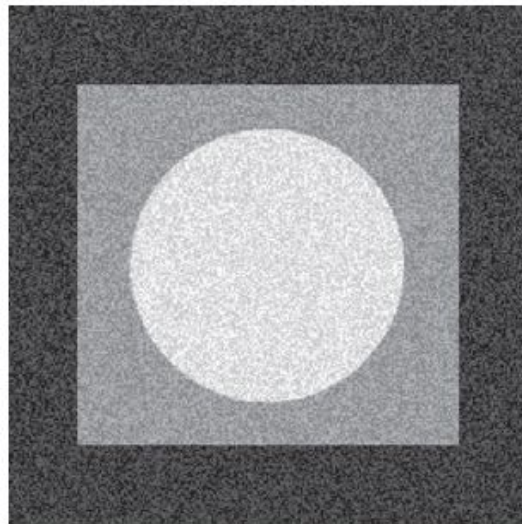
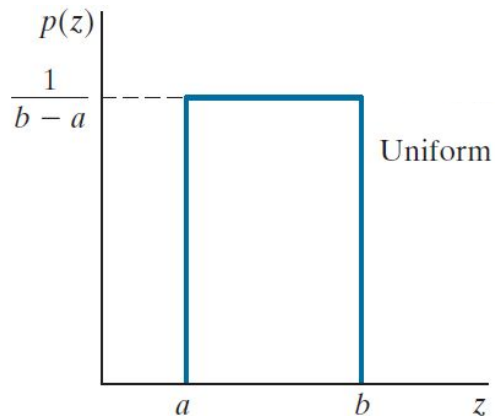
The PDF of *uniform* noise is

$$p(z) = \begin{cases} \frac{1}{b-a} & a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

The mean and variance of z are

$$\bar{z} = \frac{a+b}{2}$$

$$\sigma^2 = \frac{(b-a)^2}{12}$$



Salt-and-Pepper Noise

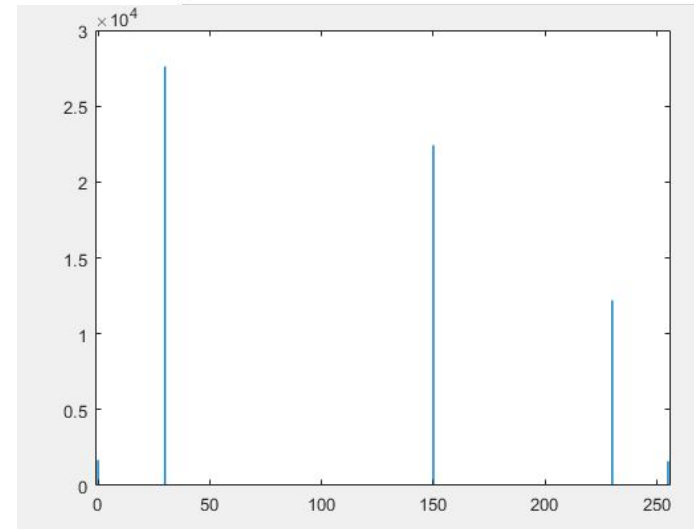
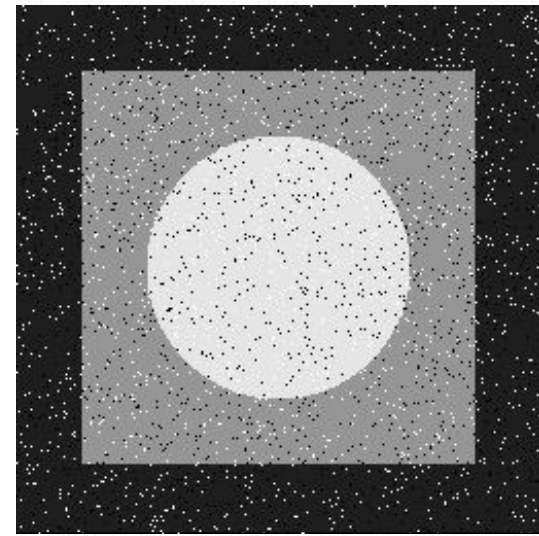
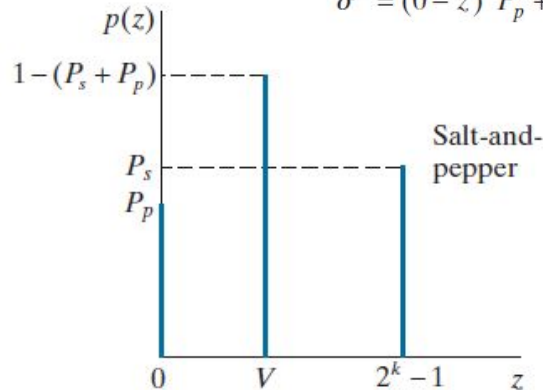
If k represents the number of bits used to represent the intensity values in a digital image, then the range of possible intensity values for that image is $[0, 2^k - 1]$ (e.g., $[0, 255]$ for an 8-bit image). The PDF of *salt-and-pepper* noise is given by

$$p(z) = \begin{cases} P_s & \text{for } z = 2^k - 1 \\ P_p & \text{for } z = 0 \\ 1 - (P_s + P_p) & \text{for } z = V \end{cases}$$

where V is any integer value in the range $0 < V < 2^k - 1$.

$$\bar{z} = (0)P_p + K(1 - P_s - P_p) + (2^k - 1)P_s$$

$$\sigma^2 = (0 - \bar{z})^2 P_p + (K - \bar{z})^2 (1 - P_s - P_p) + (2^k - 1)^2 P_s$$



Arithmetic Mean Filter

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(r, c) \in S_{xy}} g(r, c)$$

Blurring using this reduces the noise

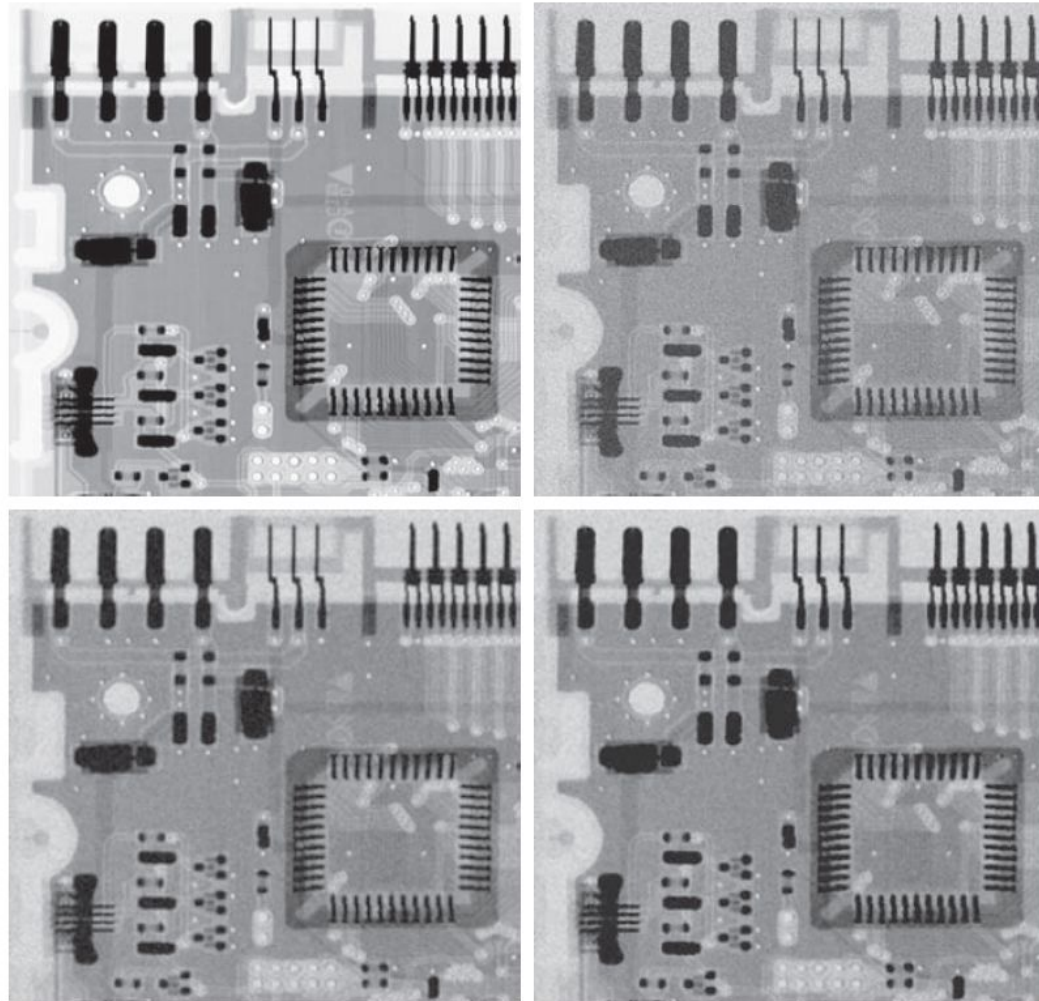
Geometric Mean Filter

$$\hat{f}(x, y) = \left[\prod_{(r, c) \in S_{xy}} g(r, c) \right]^{\frac{1}{mn}}$$

Smoothing is comparable to previous one, but it tends to lose less image detail in the process

a	b
c	d

(a) X-ray image of circuit board.
 (b) Image corrupted by additive Gaussian noise.
 (c) Result of filtering with an arithmetic mean filter of size 3×3 .
 (d) Result of filtering with a geometric mean filter of the same size. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)



Harmonic Mean Filter

$$\hat{f}(x, y) = \frac{mn}{\sum_{(r,c) \in \mathcal{S}_{xy}} \frac{1}{g(r, c)}}$$

The harmonic mean filter works well for salt noise, but fails for pepper noise.

It does well also with other types of noise like Gaussian noise.

Contraharmonic Mean Filter

This filter is well suited for reducing or virtually eliminating the effects of salt-and-pepper noise.

For positive values of Q , the filter eliminates pepper noise.

For negative values of Q , it eliminates salt noise.

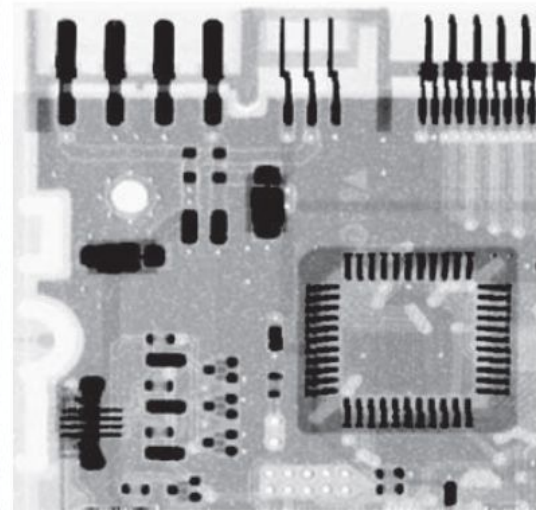
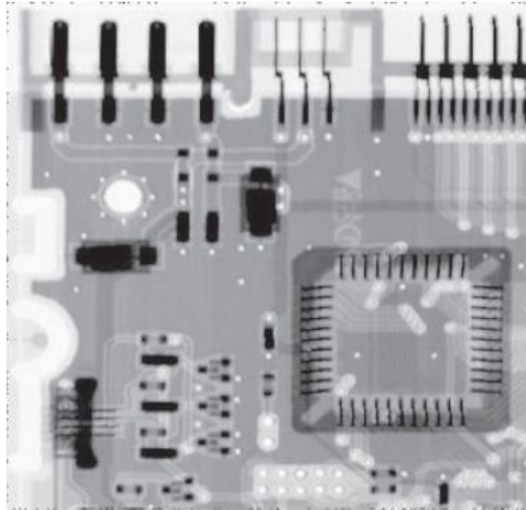
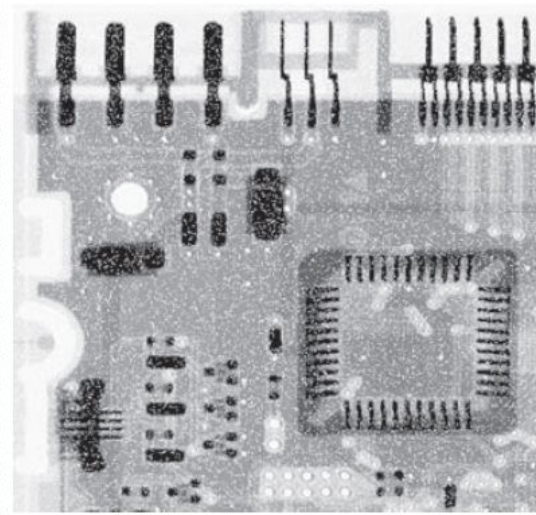
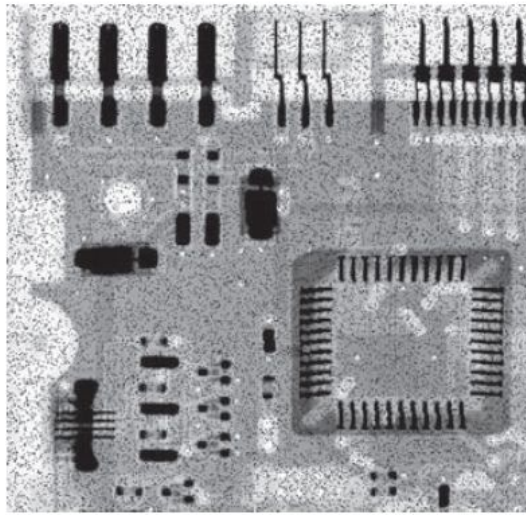
It cannot do both simultaneously.

Note that the contraharmonic filter reduces to the arithmetic mean filter if $Q = 0$, and to the harmonic mean filter if $Q = -1$.

$$\hat{f}(x, y) = \frac{\sum_{(r,c) \in S_{xy}} g(r, c)^{Q+1}}{\sum_{(r,c) \in S_{xy}} g(r, c)^Q}$$

a	b
c	d

(a) Image corrupted by pepper noise with a probability of 0.1. (b) Image corrupted by salt noise with the same probability. (c) Result of filtering (a) with a 3×3 contra-harmonic filter $Q = 1.5$. (d) Result of filtering (b) with $Q = -1.5$.



ORDER-STATISTIC FILTERS

Median Filter

$$\hat{f}(x, y) = \operatorname{median}_{(r, c) \in S_{xy}} \{g(r, c)\}$$

Work well for salt & pepper noise

Max and Min Filters

$$\hat{f}(x, y) = \max_{(r, c) \in S_{xy}} \{g(r, c)\}$$

Reduces pepper
noise

$$\hat{f}(x, y) = \min_{(r, c) \in S_{xy}} \{g(r, c)\}$$

Reduces salt
noise

Useful for finding brightest and darkest points in a given neighborhood. These filters also have application in morphological image processing.

Midpoint Filter

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(r, c) \in S_{xy}} \{g(r, c)\} + \min_{(r, c) \in S_{xy}} \{g(r, c)\} \right]$$

Combines order statistics and averaging. Works well for Gaussian and Uniform Noises.

Alpha-Trimmed Mean Filter

After trimming $d/2$ highest and $d/2$ lowest pixels

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(r,c) \in S_{xy}} g_R(r, c)$$

When $d = 0$ the alpha-trimmed filter reduces to the arithmetic mean filter discussed earlier.

If we choose $d = mn - 1$, the filter becomes a median filter.

For other values of d , the alpha-trimmed filter is useful in situations involving multiple types of noise, such as a combination of salt & pepper and Gaussian noise.

Adaptive Filtering

- Behavior changes based on statistical characteristics of the image inside the filter region defined by the rectangular neighborhood
- The price paid for improved filtering power is an increase in filter complexity

Adaptive, Local Noise Reduction Filter

Our filter is to operate on a neighborhood, S_{xy} , centered on coordinates (x, y) . The response of the filter at (x, y) is to be based on the following quantities: $g(x, y)$, the value of the noisy image at (x, y) ; σ_{η}^2 , the variance of the noise; $\bar{z}_{S_{xy}}$, the local average intensity of the pixels in S_{xy} ; and $\sigma_{S_{xy}}^2$, the local variance of the intensities of pixels in S_{xy} . We want the behavior of the filter to be as follows:

1. If σ_{η}^2 is zero, the filter should return simply the value of g at (x, y) . This is the trivial, zero-noise case in which g is equal to f at (x, y) .
2. If the local variance $\sigma_{S_{xy}}^2$ is high relative to σ_{η}^2 , the filter should return a value close to g at (x, y) . A high local variance typically is associated with edges, and these should be preserved.
3. If the two variances are equal, we want the filter to return the arithmetic mean value of the pixels in S_{xy} . This condition occurs when the local area has the same properties as the overall image, and local noise is to be reduced by averaging.

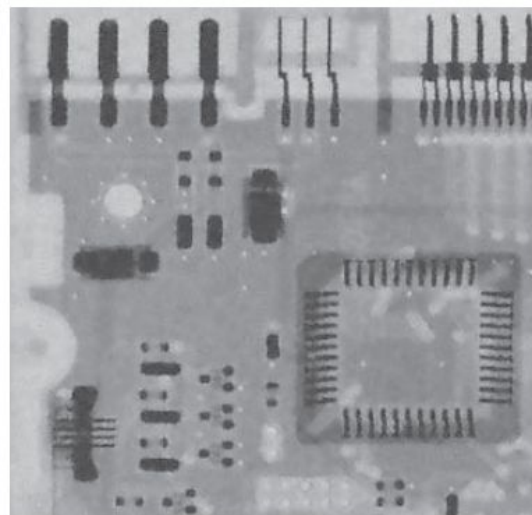
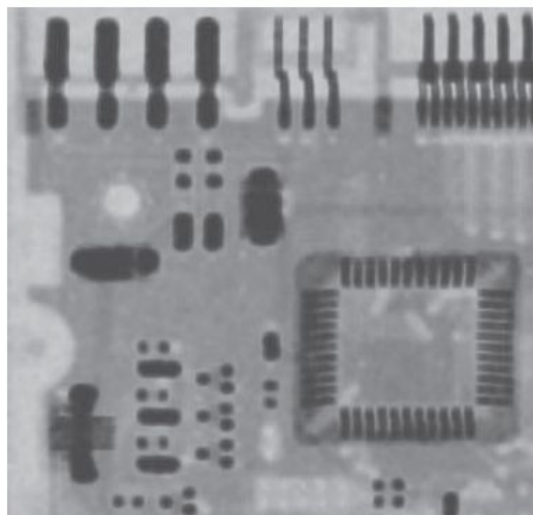
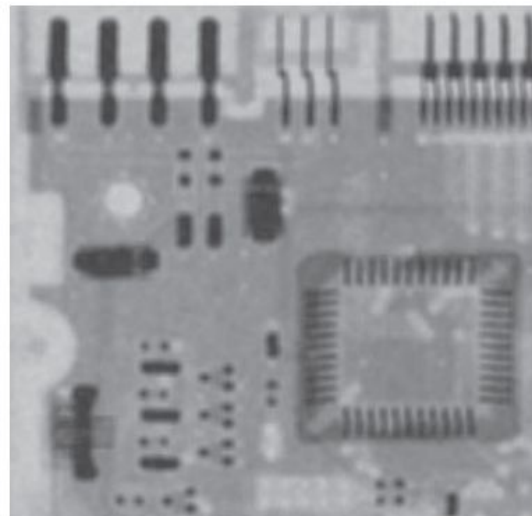
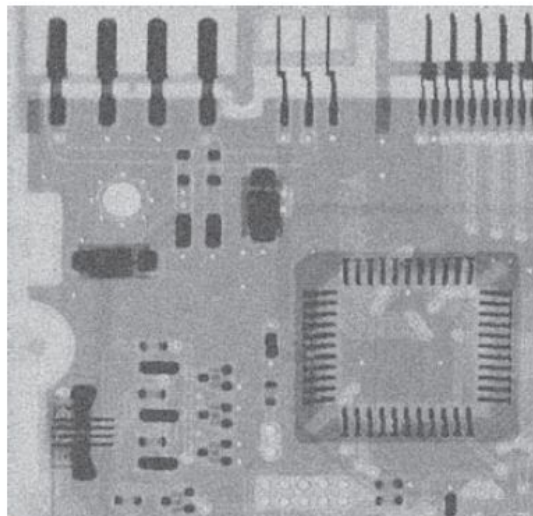
An adaptive expression for obtaining $\hat{f}(x, y)$ based on these assumptions may be written as

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_{\eta}^2}{\sigma_{S_{xy}}^2} \left[g(x, y) - \bar{z}_{S_{xy}} \right]$$

- We can estimate noise variance from a large patch of homogeneous region.
- Set the variance ratio to 1 if exceeds 1 by any chance

a	b
c	d

- (a) Image corrupted by additive Gaussian noise of zero mean and a variance of 1000.
 (b) Result of arithmetic mean filtering.
 (c) Result of geometric mean filtering.
 (d) Result of adaptive noise-reduction filtering. All filters used were of size 7×7 .



Adaptive Median Filter

Median filter performs well only in the case of low density S&P noise.

Adaptive median filter can handle higher density noise.

Variable filter size depending on conditions

Level A : If $z_{\min} < z_{\text{med}} < z_{\max}$, go to Level B

Else, increase the size of S_{xy}

If $S_{xy} \leq S_{\max}$, repeat level A

Else, output z_{med} .

Level B : If $z_{\min} < z_{xy} < z_{\max}$, output z_{xy}

Else output z_{med} .

$z_{\min}$ = minimum intensity value in S_{xy}

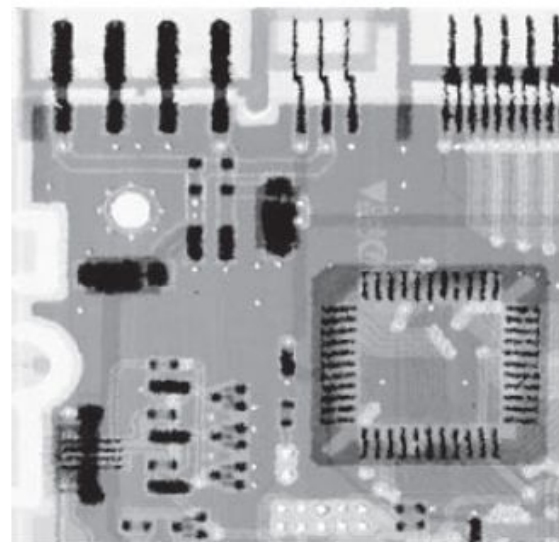
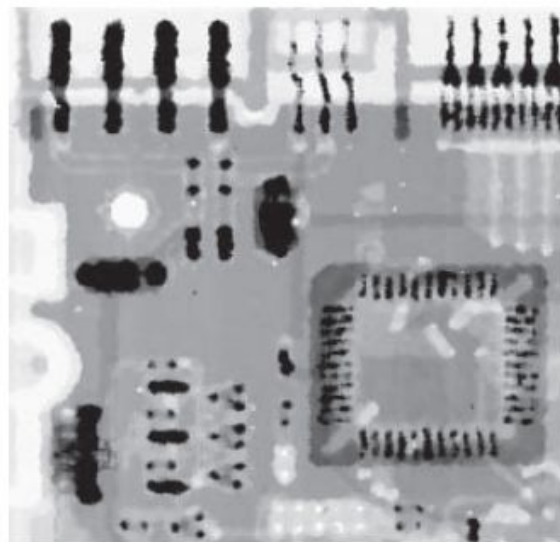
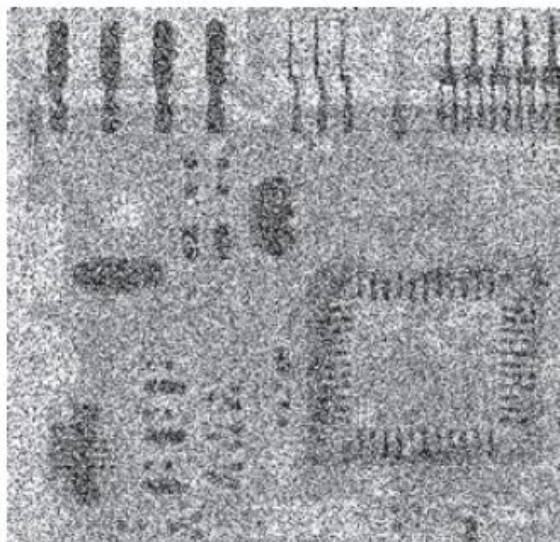
$z_{\max}$ = maximum intensity value in S_{xy}

z_{med} = median of intensity values in S_{xy}

z_{xy} = intensity at coordinates (x, y)

$S_{\max}$ = maximum allowed size of S_{xy}

- Level B ensures that the pixel is replaced with the median only when the pixel is salt or pepper.
- Level A protects us from entering Level B if the noise is dense or if the original pixels are also extremes. It attempts to increase the filter size to reduce the noise density. If unable to reach Level B even after reaching maximum filter size, it outputs median.



a b c

(a) Image corrupted by salt-and-pepper noise with probabilities $P_s = P_p = 0.25$. (b) Result of filtering with a 7×7 median filter. (c) Result of adaptive median filtering with $S_{\max} = 7$.
